

Software Engineering II

Designphase

Mathis Neunzig
ABAP Developer at SAP
mathis.neunzig@gmail.com



Disclaimer

- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesung sind kein Ersatz für Literatur und können nicht als Quellen für wissenschaftliche Arbeiten genutzt werden.
- Diese Präsentation sowie alle weiteren Materialien dieser Vorlesungen dürfen nur von den Studierenden des Kurses WI 2024 SE A zur Nachbereitung der Vorlesung genutzt werden. Eine kommerzielle Nutzung und Weitergabe ist verboten.
- Alle Präsentation sowie alle weiteren Materialien werden vor Beginn der jeweiligen Vorlesung hochgeladen.
- Zur besseren Lesbarkeit wird auf die gleichzeitige Nutzung von weiblichen & männlichen Begriffen verzichtet. Es wird das generische Maskulinum verwendet. Dabei sind weibliche und sonstige Formen des Wortes gleichwertig mitgemeint. (Beispiel: statt "Kund:innen" → „Kunden“)

Modularchitektur / Modulspezifikation

Was ist ein Modul?

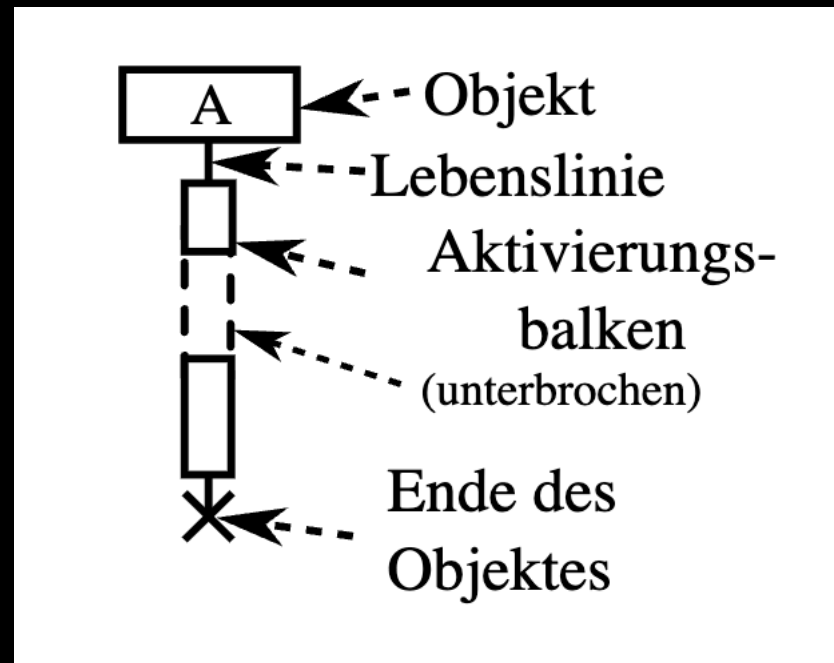
Wie modelliere ich ein einzelnes Modul?

Was sind Entwurfsmuster?

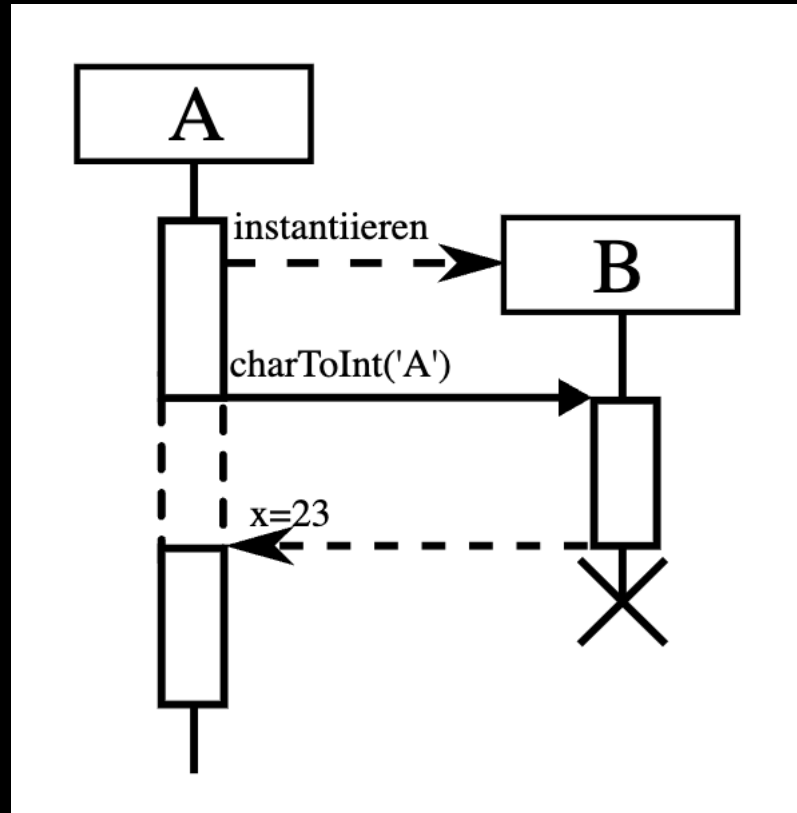
Modulspezifikationen

- Was ist ein Modul?
 - Modul $\neq$ Klasse
 - Modul kann Klasse oder Bestandteil einer Klasse sein
 - Modulgröße in Zeilen ist irrelevant
 - Module sind nicht zwingend wiederverwendbar
- Beispiel
 - FileReader
 - FileWriter
 - Datenbankschnittstelle
 - Frontend
 - API-Schnittstelle

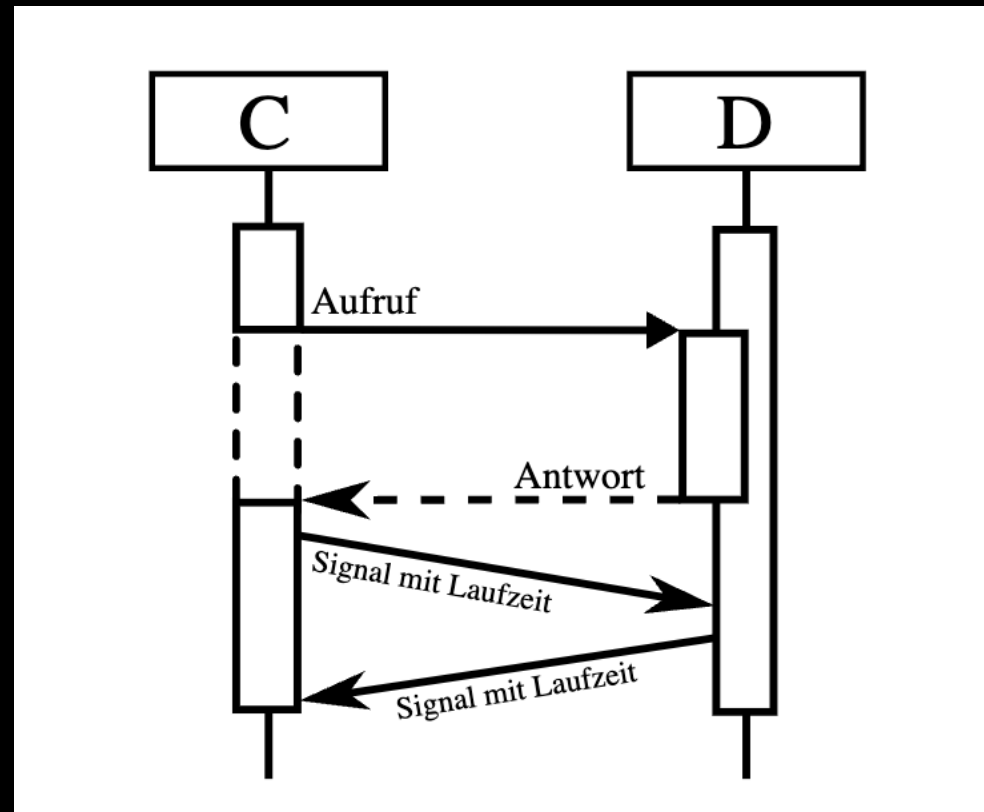
UML Sequenzdiagramm



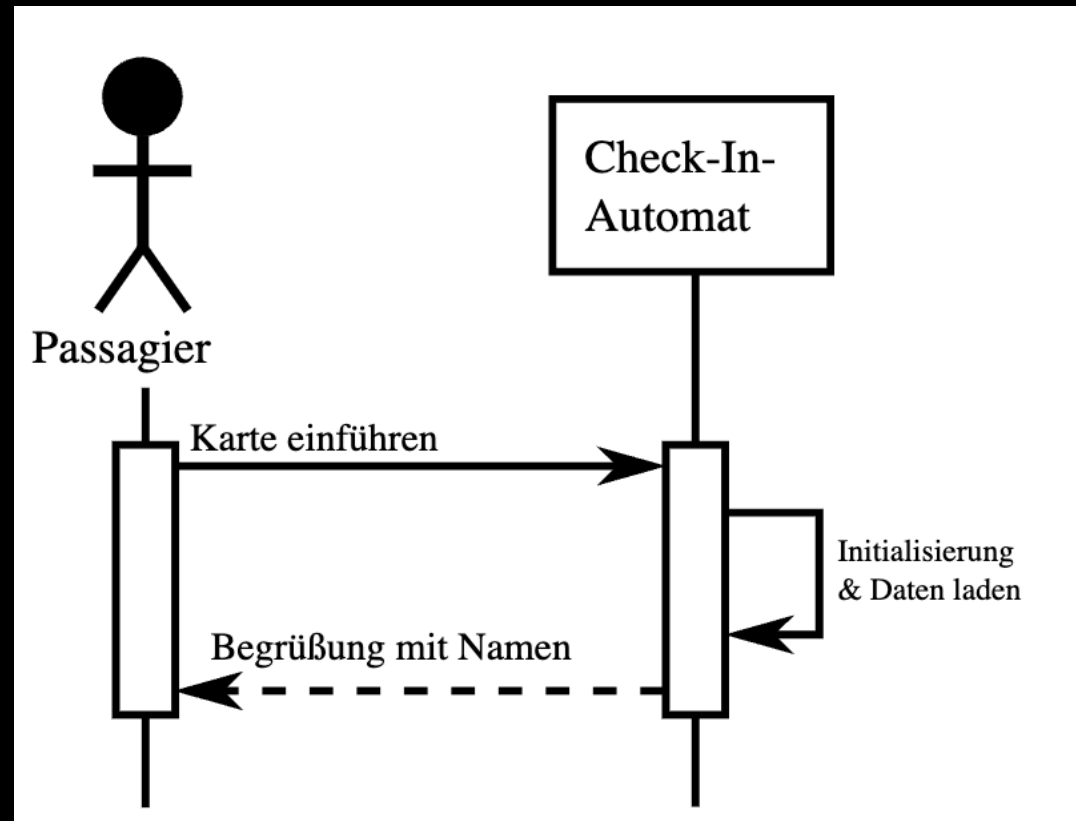
UML Sequenzdiagramm



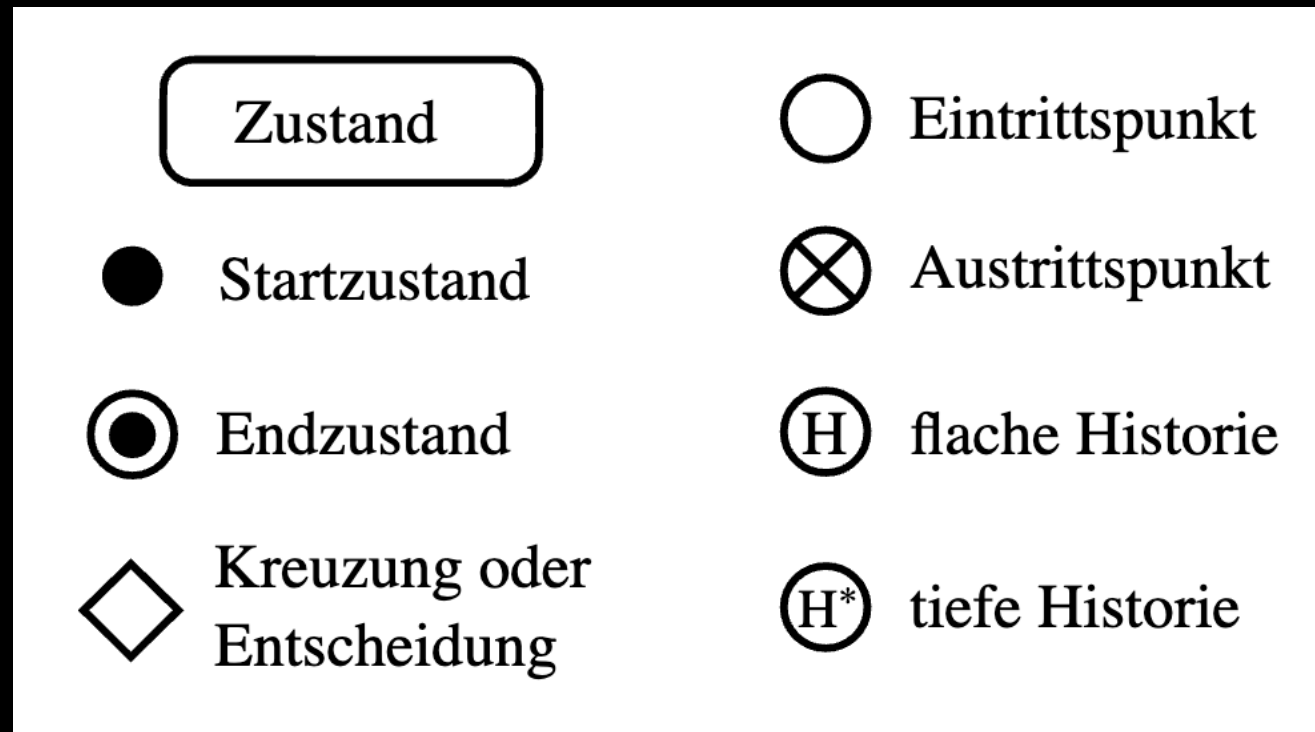
UML Sequenzdiagramm



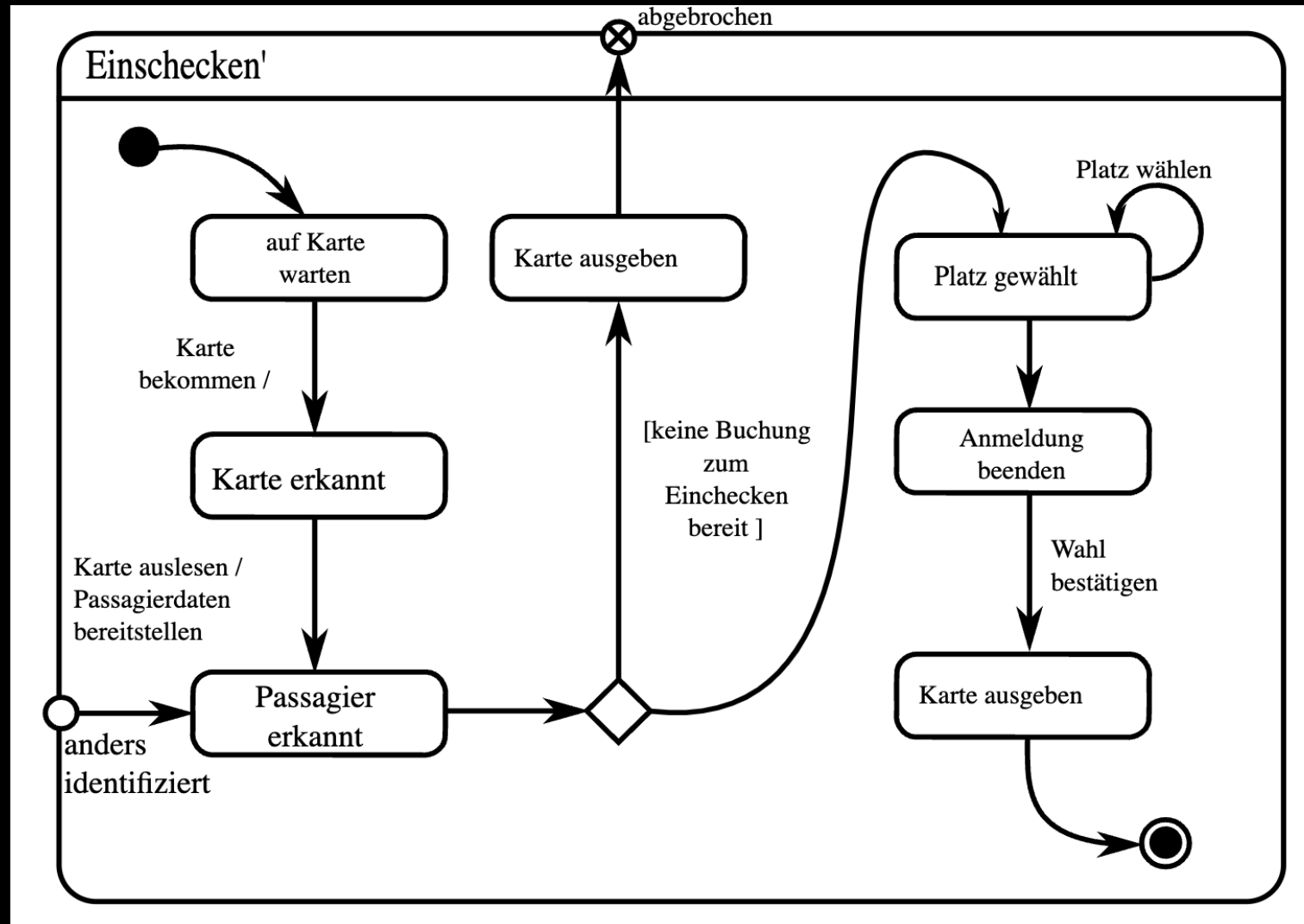
UML Sequenzdiagramm



UML Zustandsdiagramm



UML Zustandsdiagramm



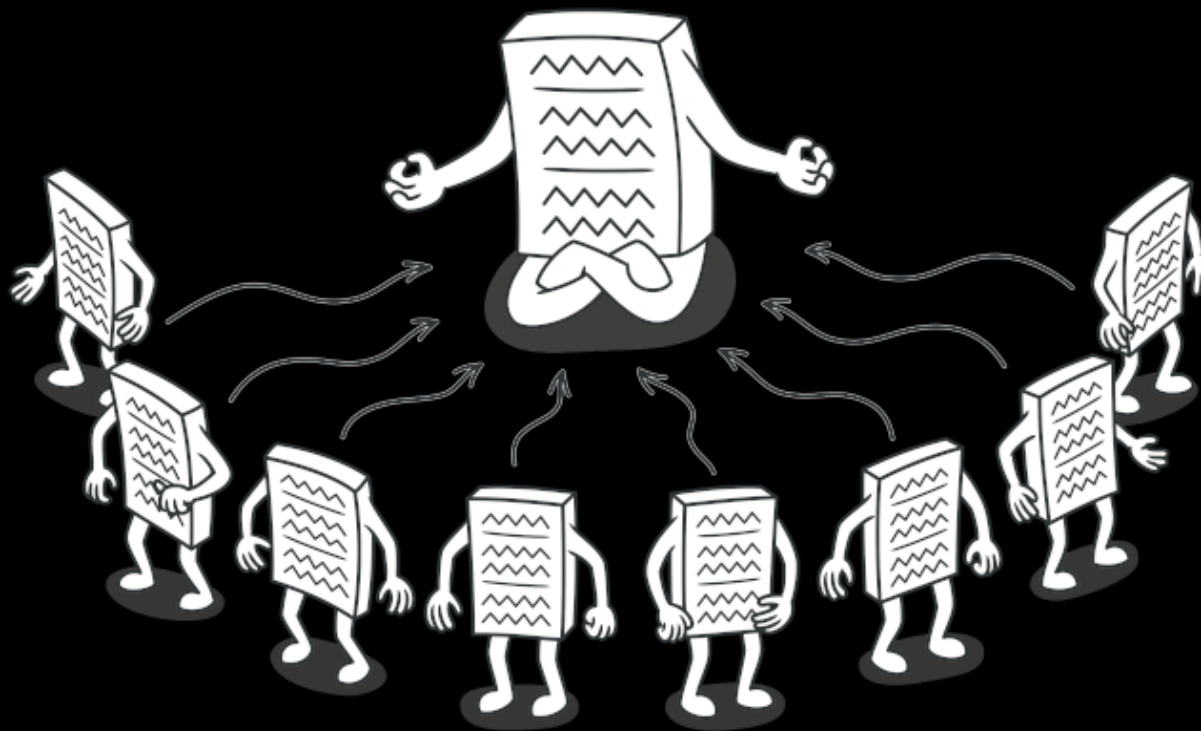
Modulspezifikationen

- Entwurfsmuster
 - Erzeugungsmuster
 - Strukturmuster
 - Verhaltensmuster
 - Objektrelationale Muster
 - Nachrichtenübermittlungsmuster
- Anti-Muster
- Mockups

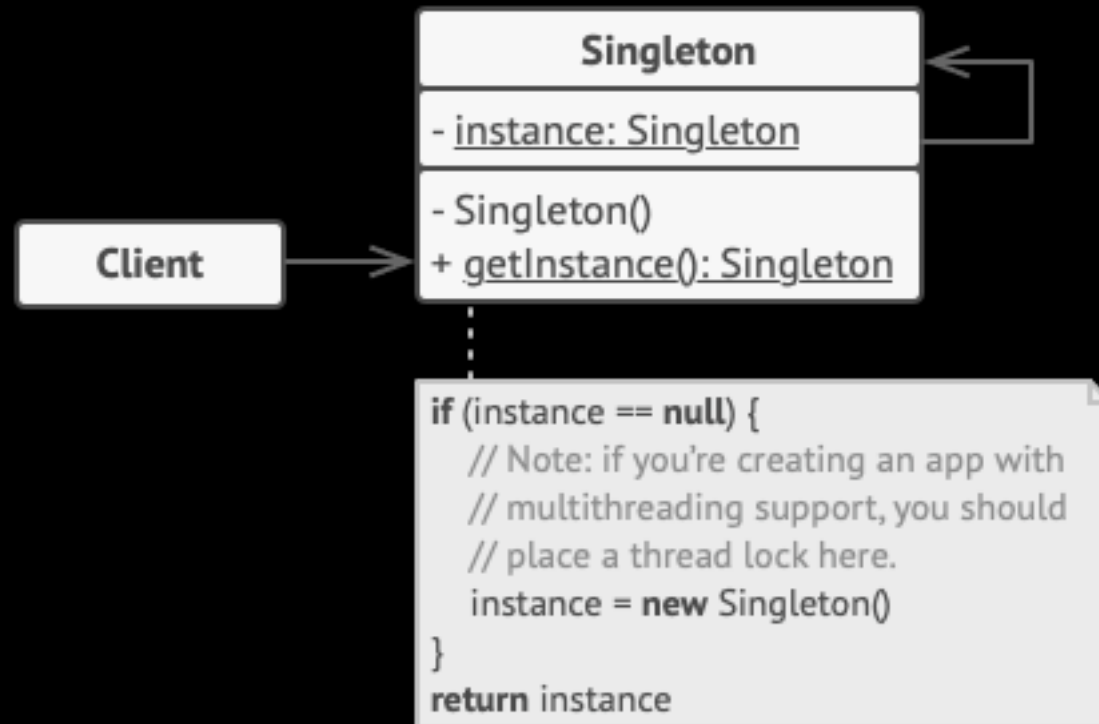
Erzeugungsmuster

- Erbauer (Builder)
- Fabrikmethode (Factory method)
- Multiton (Multiton)
- Prototyp (Prototype)
- Singleton (Singleton)
(Disclaimer: Wissenschaftler sind sich uneinig, ob Singleton ein Entwurfsmuster oder ein sogenanntes „Anti-Pattern“ ist ⚠)

Singleton



Singleton



Factory



Factory

Ich erstell' mir
mal Katzen

© Main.java

- cats: Cat[]

+ main(): void

+ createCat(): Cat

Factory

Ich erstell' mir
mal Katzen

© Main.java

- cats: Cat[]

+ main(): void

+ createCat(): Cat



Factory

Ich erstell' mir
mal Katzen

© Main.java

- cats: Cat[]

+ main(): void

+ createCat(): Cat



Factory

Ich erstell' mir
mal Katzen

© Main.java

- cats: Cat[]

+ main(): void

+ createCat(): Cat



Factory

Ich erstell' m
mal Katzen

© Main.java

- cats: Cat[]

main(): void
+ createCat(): Cat



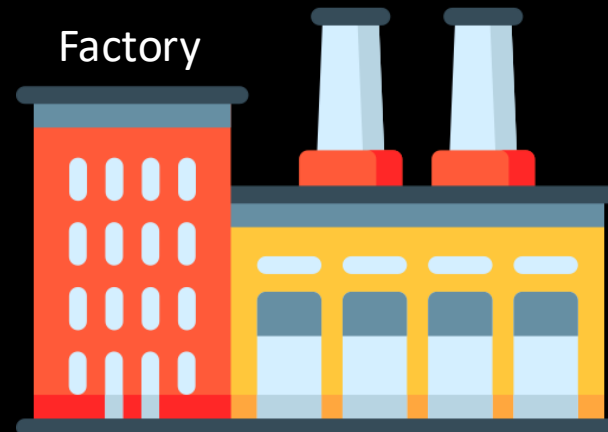
Factory

Dann bestell ich
halt Katzen...

© Main.java

- cats: Cat[]

+ main(): void

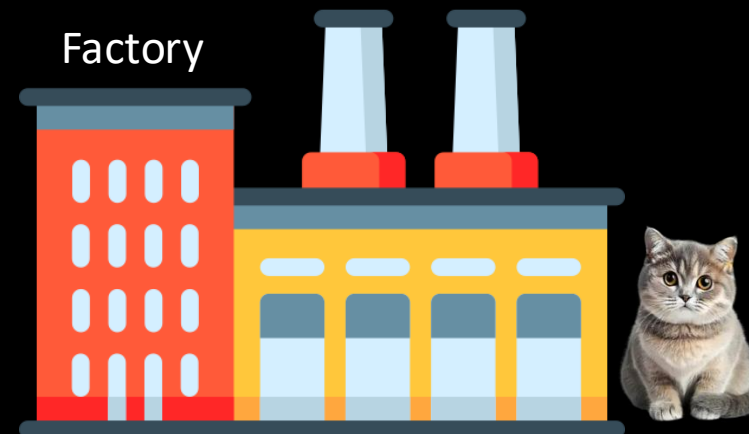


© CatFactory.java

+ create(): Cat

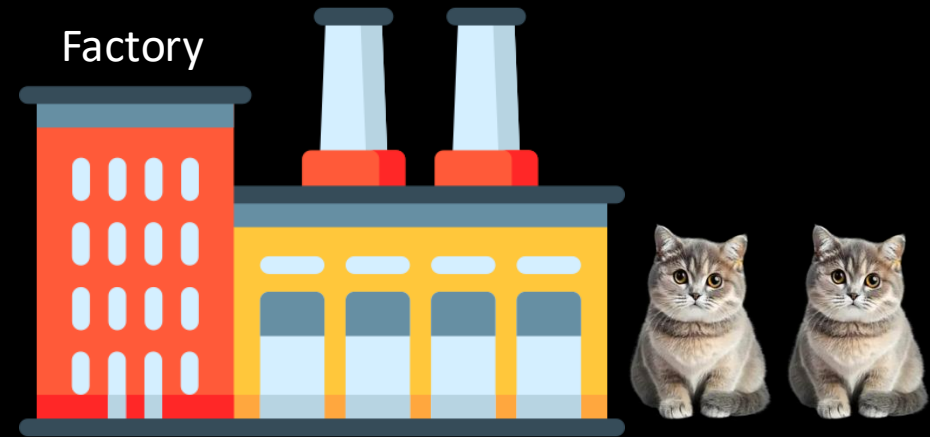
Factory

Dann bestell ich
halt Katzen...

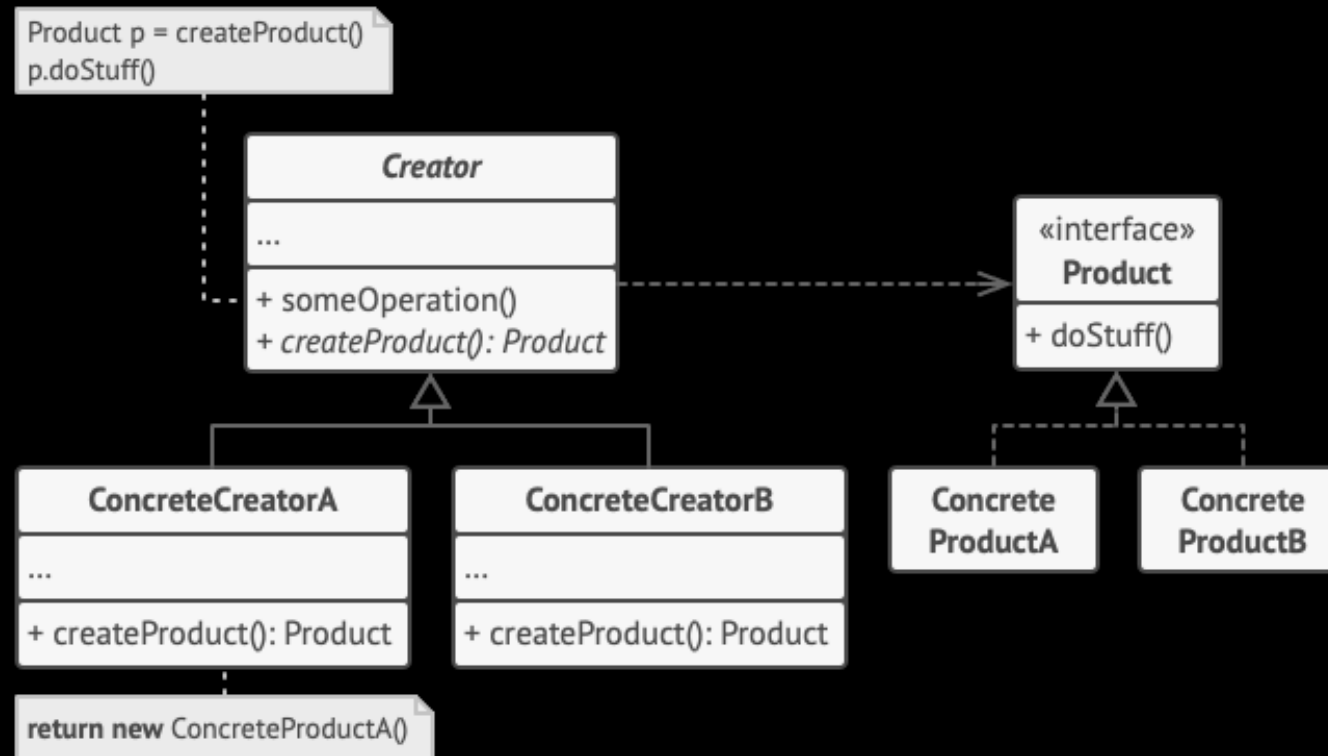


Factory

Dann bestell ich
halt Katzen...



Factory



Strukturmuster

- Container (Container)
- Fassade (Facade)
- Dekorierer (Decorator)
- Fliegengewicht (Flyweight)
- Stellvertreter (Proxy)
- Kompositum (Composite)

Decorator



Decorator

Preis: 5€



OrangeCatDecorator



Decorator

OrangeCatDecorator

Preis +1€



Decorator

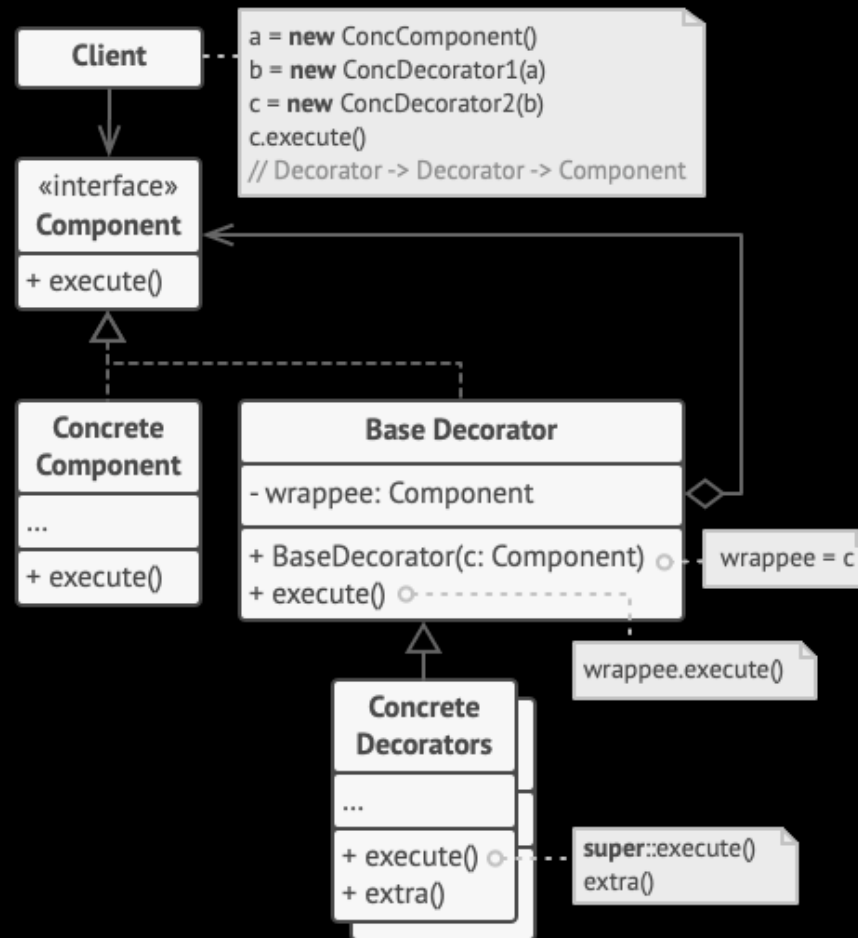
OrangeCatDecorator



Preis: 6€



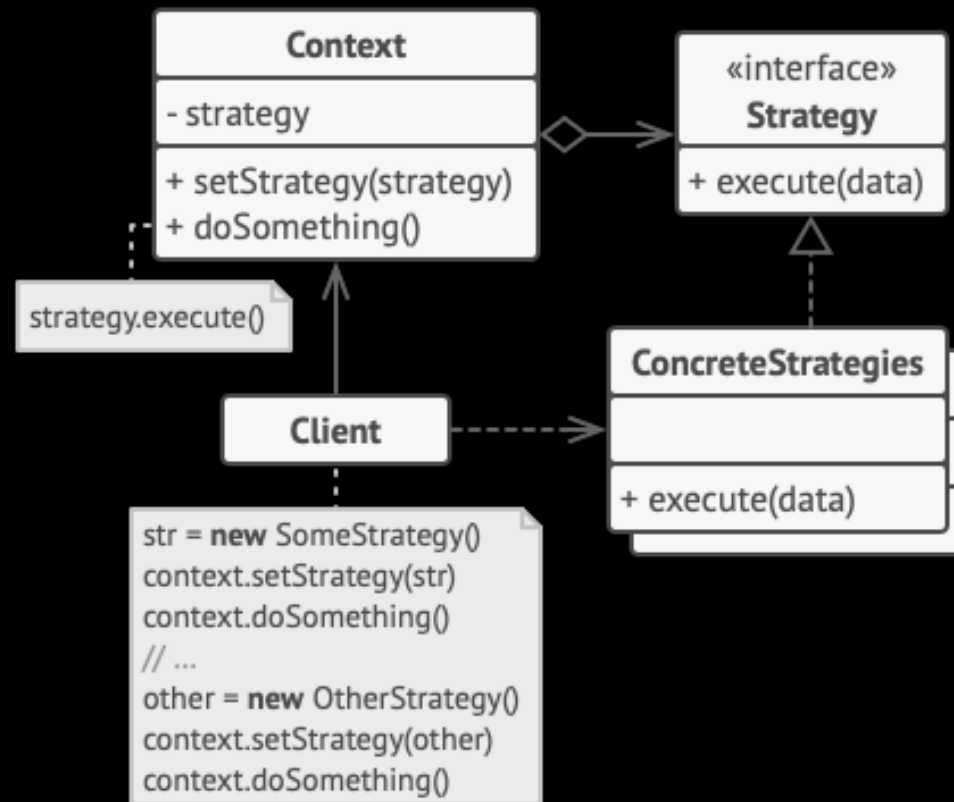
Decorator



Verhaltensmuster

- Iterator (Iterator)
- Strategie (Strategy)
- Kommando (Command)
- Beobachter (Observer)
- MapReduce (MapReduce)
- Nullobjekt (Null Object)
- Schablone (Template)

Strategy



Literaturhinweise

The Catalog of Design Patterns

Creational patterns

These patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.



Factory Method



Abstract Factory



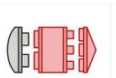
Builder



Prototype

Structural patterns

These patterns explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient.



Adapter



Bridge



Composite



Decorator

Behavioral patterns

These patterns are concerned with algorithms and the assignment of responsibilities between objects.



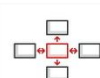
Chain of Responsibility



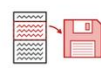
Command



Iterator



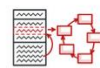
Mediator



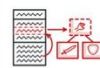
Memento



Observer

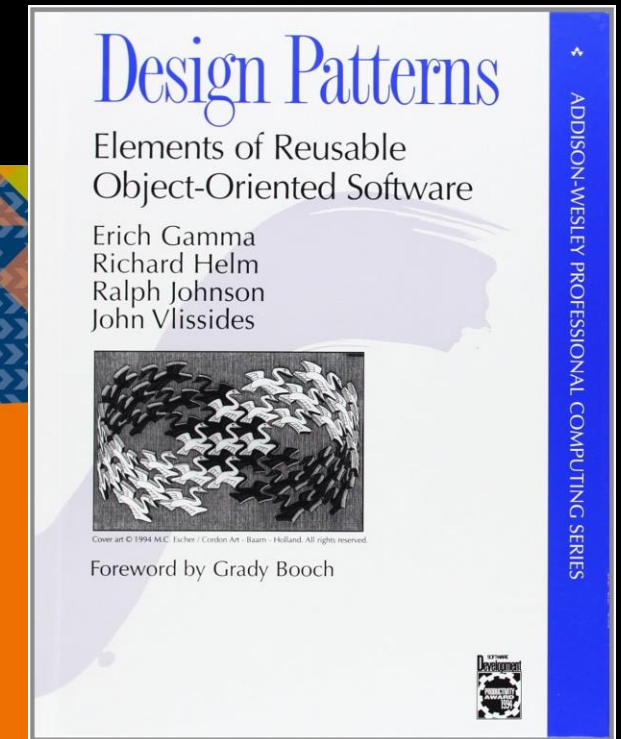


State



Strategy

<https://refactoring.guru/design-patterns/catalog>



Anti-Pattern

- Zugunglück (Train Wreck)
- Gottobjekt (God Object)
- Gasfabrik (Gas Factory)
- Großer Matschklumpen (Big Ball of Mud)
- Zwiebel (Onion)
- Kopieren und Einfügen (Copy and Paste Programming)

Train Wreck

```
String.getLength()  
    .subtract(3)  
        .add(5)  
            .pow(10)  
                .toString()  
                    .substring(2, 2)  
                        .convertToCar()  
                            .startCar();
```

```
Sondern: <Class>.getCar(name);
```

Übung: Entwurfsmuster

- Sucht euch ein Entwurfsmuster und eine Programmiersprache aus
- Zu zweit (oder auch zu dritt)
- Implementiert euer Entwurfsmuster (siehe Repository)
- Erstellt ein Plakat / OnePager, um dem Kurs das Pattern zu zeigen
 - Steckbrief (Name, Art, etc.)
 - Vorteile / Nachteile
 - Einsatzgebiete
 - Warum existiert es?
 - Beispiel

Übung: Entwurfsmuster

Erzeugungsmuster (Creational)	Namen	Verhaltensmuster (Behavioral)	Namen	Strukturmuster (Structural)	Namen
Abstract Factory		Memento		Proxy	
Prototype		Visitor		Composite	
Builder		Observer		Bridge	
		Iterator		Adapter	
		Chain of Responsibility			

Danke!

Mathis Neunzig
mathis.neunzig@gmail.com

